

MINIFLIX: SCALABLE OTT ARCHITECTURE

A High-Concurrency, Low-Latency Video Streaming System Design

Project Focus: Edge Caching & Scalability

Target Concurrency: 100,000+ Active Users

Architecture: Cloud Distributed CDN

1. PROBLEM STATEMENT & CHALLENGE

When a centralized video platform experiences sudden spikes in concurrent traffic (such as 100,000+ users attempting to stream popular content simultaneously during peak hours), standard monolithic application servers fail due to network congestion, bandwidth exhaustion, and database query bottlenecks. The core objective of this project is to architect an engineering solution that delivers uninterrupted, high-definition video playback without server degradation or streaming latency.

2. SYSTEM ARCHITECTURE & CORE COMPONENTS

MiniFlix addresses high-concurrency video delivery through a distributed four-pillar system design inspired by enterprise media platforms like Netflix and Amazon Prime Video:

COMPONENT	TECHNOLOGY STACK	ENGINEERING ROLE & FUNCTION
Cloud Storage	AWS S3 / Google Cloud Storage	Stores raw master media files, video segments (HLS/DASH), and encrypted asset bundles securely off-premise.
Content Delivery Network (CDN)	AWS CloudFront / Cloudflare	Caches static video segments at edge locations geographically closer to users to eliminate long-distance network hop latency.
In-Memory Cache Layer	Redis / Memcached	Caches viral video metadata, trending Top-10 lists, and session keys to reduce direct database load by up to 90%.
Load Balancer & Auto-Scaler	AWS ALB / NGINX / Node.js Cluster	Distributes incoming API traffic dynamically across scalable microservice instances to prevent single-server bottlenecks.

3. END-TO-END REQUEST & DATA FLOW

Step 1: User Request Initiation

The user clicks 'Play' on the MiniFlix web/mobile application frontend (HTML5/React player).

Step 2: Edge CDN Evaluation

The request routes to the geographically nearest Edge Location (CDN Node). If the video chunks are cached locally (Cache Hit), playback starts instantly (<50ms).

Step 3: Load Balanced API Routing

On a CDN Cache Miss, the traffic is directed through an Application Load Balancer to an active, under-utilized backend microservice worker.

Step 4: Fast Metadata Lookup via Redis

The backend queries the Redis in-memory cache layer instead of hitting the core Mongo/SQL database, returning manifest metadata in under 10ms.

Step 5: AWS S3 Origin Fetch & Caching

Media chunks are fetched from AWS S3, streamed to the user, and saved in the CDN edge cache for all subsequent requests from that region.

4. KEY ARCHITECTURAL TAKEAWAY & PRESENTATION PITCH

KEY PRESENTATION PITCH (WINNING TECHNICAL ARGUMENT)

"By implementing Edge Caching via CDNs alongside a Redis In-Memory Caching Layer, MiniFlix offloads over 90% of read traffic away from the central origin server. This ensures ultra-low latency, drastically cuts bandwidth overhead, and allows the system to effortlessly sustain 100,000+ concurrent viewers without performance degradation—mirroring real-world architectures used by platforms like Netflix Open Connect."